

# prog2 lab B1

array e strutture,statiche e dinamiche

# Ricapitoliamo

- Possiamo avere strutture dati di diversi tipi, qui parliamo di **array** e **struct**
- Array e' una sequenza di N elementi omogenea (dello stesso tipo)
- i singoli elementi non hanno nome ma una **distanza** dall'origine(indice da 0 a N-1)
- Struct e' una sequenza di elementi di qualunque tipo
- i singoli elementi non hanno un indice ma un **nome**
- array e struct si possono annidare a piacere, si puo' creare ad es un array di struct che a loro volta contengono array e cosi' via

# Creazione/assegnazione

- possiamo definire un array o una struct allocata nella memoria statica (dal compilatore) o in quella dinamica (a runtime tramite malloc() ) da noi.
- se il dato e' allocato staticamente conosciamo gia' l'indirizzo (il nome della variabile)
- se e' dinamico memorizzeremo **l'indirizzo** in una variabile di tipo puntatore (nome\*)
- mentre si puo' fare assegnazione tra due struct dello stesso tipo, per ragioni storiche non si puo' fare tra array

# Utilizzo array

```
int staticarray[10]; // dimensione conosciuta a compile time
```

```
int * dynamicarray; // dimensione a runtime
```

```
...
```

```
staticarray[5]=3;
```

```
dynamicarray = malloc(sizeof(int)*10);
```

```
dynamicarray[4]=1;
```

```
free(dynamicarray); // se statico da' errore!
```

```
dynamicarray = NULL;
```

```
// nel punto di utilizzo i due sono indistinguibili
```

```
// il programmatore C deve tener conto della provenienza
```

# Utilizzo struct

```
typedef struct {int m1;} s;  
s staticstruct;  
s* dynamicstruct;  
...  
staticstruct.m1 =55;  
dynamicstruct = malloc(sizeof(s));  
dynamicstruct->m1 =66;  
free(dynamicstruct);  
dynamicstruct = NULL;
```

# array, struct e funzioni di utility

capita quasi sempre che, a meno di programmi molto semplici, ogni struct o array abbia bisogno di alcune funzioni di base:

- creazione
- distruzione
- conversione da/a stringa

conviene allora pensarle fin dall'inizio, rendono il codice omogeneo e piu' leggibile (pensate ad esempio alle stampe di debug)

# attenti a parametri array e struct!

quando si passa un array, si passa sempre l'indirizzo dell'array di partenza, non si può passare una copia di tutto l'array

nel chiamato ogni modifica sarà visibile al ritorno dalla chiamata

quando passo una struct, ho due casi:

se passo un puntatore siamo come nel caso array di prima

se passo una struct per valore, creo una copia intera e le modifiche non toccano l'originale

# Passare quindi per valore o referenza?

- Ovviamente ogni caso specifico richiede valutazione
- ma, a parita' di tutto, che scegliere?
- Per omogeneita' le struct si possono passare sempre per referenza (puntatori a struct) anche quando sono piccole, con l'assunzione che possono essere modificate
- gli array sono gia' puntatori quindi vanno bene cosi'
- in questo modo i due tipi di dato sono trattati nello stesso modo
- se non indispensabile evitate **&** (address\_of) a meno di trovarvi in programmi molto piccoli

# Passare per valore o referenza?

- in C comunque i parametri in se' sono sempre passati per VALORE, posso al pero' trattarli come indirizzo e modificare il contenuto a quell'indirizzo,
- trattare tutto come puntatori ci avvicina alla convenzione dei linguaggi ad oggetti, dove tutto e' un oggetto (una struct arricchita) e ci si passa avanti ed indietro sempre riferimenti agli oggetti.
- il codice diventa cosi' piu' omogeneo e rapido da leggere
- vediamo il sorgente in esempio1:
- L2\_SRC/L2\_Esempio1/**parametri.c**

# la malloc()

conviene per omogeneita' per ogni struct{} avere due funzioni  
crea...(), libera..() :

```
ptr_to_mia  crea(.....){  
ptr_to_mia retp =malloc(sizeof(mia));  
// inizializza in base ai parametri ..... tutti gli attributi  
return retp; }
```

```
void libera_mia(ptr_to_mia p){  
// libera tutti attributi allocati etc  
free(p);} 
```

# la malloc()

- in questo modo renderete piu' leggibile il codice e siete invogliati a non rinunciare per pigrizia all'allocazione dinamica
- Come gia' detto, quasi sempre conviene investire tempo a dotare una struttura dati di tutte le funzioni di base

# la free()

- Come sappiamo, se non si chiama la free() poco a poco la memoria si riempie
- Purtroppo se si la si chiama in modo errato (ad esempio una struct ha due puntatori), la memoria risulta libera, viene riusata ma il secondo puntatore e' ancora in uso!!! e si generano degli errori oscuri, difficili da investigare
- Uno strumento 'salvavita' e' valgrind  
<https://valgrind.org/>  
che rileva moltissimi errori nella gestione della memoria

# errori della free()

Quando deallochiamo strutture, e' prudente non deallocare "troppo"

Si rischia di deallocare oggetti usati altrove, causando errori difficili da individuare.

Se liberate troppo poco, riempite la memoria, ma l'errore e' piu' evidente.

La **free()** puo' essere usata **solo** su puntatori creati dalla **malloc()**

quando ho una struct{} posso fare la free solo di puntatori creati da malloc()

guardiamo la slide seguente

# errori della free()

```
typedef struct p persona , *personaPtr;  
struct p {char *name; int eta; char *cf; };
```

```
personaPtr io = malloc(sizeof(persona));  
io->nome = "maria"; // no malloc()  
io->eta = 22; // no malloc()  
io->cf = malloc(15*sizeof(char));
```

.....

```
free(io->nome); // errore!  
free(io->eta); // errore!  
free(io->cf); // OK  
free(io); // OK
```

# la realloc()

Se abbiamo allocato ad es un array di 100 elementi

```
int ** myarray = malloc(sizeof(int)*100);
```

ma vogliamo aumentarne la capacita' a 200 , dovremmo fare

```
int ** newarray = malloc(sizeof(int)*200);
```

```
memcpy(newarray,myarray,sizeof(int)*100); // copia
```

```
free(myarray);
```

```
myarray = newarray;
```

la realloc() fa tutto questo per noi!

```
myarray = realloc(sizeof(int)*200); // errore non controllo NULL!!
```

come malloc() anche realloc() puo' dare NULL

# uso corretto di realloc()

Se abbiamo allocato ad es un array di 100 elementi

```
int ** myarray = malloc(sizeof(int)*100);
```

.....

// raddoppio tipicamente

```
int ** newarray = realloc(sizeof(int)*100*2);
```

```
if (newarray != NULL){
```

```
    myarray = newarray;
```

```
} else {
```

```
    // errore! dovete provvedere...
```

```
;
```

```
}
```

# strutture a dimensione fissa o variabile?

dipende ovviamente dai requisiti.

- In linea di principio **preferite strutture dinamiche**, così il programma non avrà mai limitazioni derivanti dall'implementazione ma solo dalla memoria del computer su cui gira, anche se usare malloc() e realloc() è più laborioso (e un po' più lento).
- Se foste costretti ad ottimizzare potreste anche riservare una serie di struct preallocate in un array, ed usare malloc() quando superate le richieste

# idea generale

```
typedef struct {.....} miastruct;  
miastruct starter[1000]; // prime 1000 senza malloc()  
int used =0;
```

```
miastruct * get(){  
    if (used < 1000){  
        return &starter[used++];  
    }  
    else  
        return malloc(sizeof(miastruct));  
}
```

```
// Non e' tutto cosi' semplice ovviamente, che fare con la free()?  
// etc etc
```

# Esempio

il nel folder esempio2 il file **mialloc.c** contiene un esempietto col solito calcolo dei tempi